Работа 4. Управление ресурсами: правило трёх

Тема: Владение ресурсами. Конструктор копирования. Оператор присваивания. Правило трёх.

Цель работы: понять, как класс управляет динамической памятью. Научиться писать конструктор копирования и оператор присваивания с глубоким копированием. Освоить защиту от самоприсваивания и возврат ссылки для цепочек (например, a=b=c).

Предметная область

Класс Matrix из Работы 3 (динамическая двумерная матрица int с выделением памяти через new). Если у вас реализация отличается от эталонной — приведите её к виду, где есть поля int** data_, int rows_, int cols_ и деструктор, освобождающий память.

Что нужно реализовать

1. Конструктор копирования Matrix(const Matrix& other) — глубокое копирование: создаёт новый массив и копирует значения, а не указатели.
2. Оператор присваивания Matrix& operator=(const Matrix& other) — глубокое копирование с предварительным освобождением старой памяти.

Требования:

- Конструктор копирования выделяет новую память и копирует данные поэлементно.
- operator= сначала освобождает старую память, потом копирует новые данные.
- В operator= — защита от самоприсваивания: if (this == &other) return *this;
- operator= возвращает Matrix& для поддержки цепочек вида d = e = a;
- Оба метода корректно обрабатывают пустую матрицу (0×0): не падают, не выделяют память.

Демонстрационная программа (main.cpp)

```
#include <iostream>
#include "Matrix.h"

int main() {
    // 1. Создать матрицу a(3, 4), заполнить, вывести
    Matrix a(3, 4);
    a.fillRandom();   // метод из Работы 3
    std::cout << "Matrix a:\n";
    a.print();
```

```
    // 2. Копирование через конструктор копирования
    Matrix b = a;

    // 3. Изменить b и убедиться, что a не изменилась
    b.set(0, 0, 999);
    std::cout << "\n After b.set(0, 0, 999):\n";
    std::cout << "Matrix a:\n";
    a.print();
    std::cout << "Matrix b:\n";
    b.print();

    // 4. Создать c(2, 2), заполнить, присвоить a
    Matrix c(2, 2);
    c.fillRandom();
    c = a;

    // 5. Вывести c — должна быть копия a
    std::cout << "\nMatrix c after c = a:\n";
    c.print();

    // 6. Цепочка присваиваний
    Matrix d, e;
    d = e = a;
    std::cout << "\nMatrix d:\n"; d.
    print();
    std::cout << "Matrix e:\n";
    e.print();

    // 7. Самоприсваивание
    a = a;
    std::cout << "\nMatrix a after a = a:\n";
    a.print();
    std::cout << "(should be unchanged)\n";

    return 0;
}
```

Примеры фрагментов кода

Фрагменты не копируйте полностью — они даны как подсказка, а не как готовое решение. Попробуйте сначала написать самостоятельно.

Конструктор копирования:

```
Matrix::Matrix(const Matrix& other)
```

```
    : data_(nullptr), rows_(other.rows_), cols_(other.cols_) {
    if (rows_ == 0 || cols_ == 0) {
        rows_ = 0;
        cols_ = 0;
        return;
    }
    data_ = new int*[rows_];
    for (int i = 0; i < rows_; ++i) {
        data_[i] = new int[cols_];
        for (int j = 0; j < cols_; ++j) {
            data_[i][j] = other.data_[i][j];
        }
    }
}
```

Оператор присваивания — структура (допишите реализацию):

```
Matrix& Matrix::operator=(const Matrix& other) {
    if (this == &other) {
        return *this;   // защита от самоприсваивания
    }
    // 1. Освободить старую память (цикл по строкам, потом delete[] data_)
    // 2. Скопировать rows_ и cols_ из other
    // 3. Если other пустая — обнулить и вернуть *this
    // 4. Выделить новую память и скопировать данные
    return *this;
}
```

Обратите внимание: operator= возвращает Matrix& и принимает const Matrix&. Деструктор из Работы 3 должен корректно освобождать память — проверьте, что он не вызывается дважды для одного и того же участка памяти (подумайте, какие действия в этой работе гарантируют, что этого не произойдет - перечислите в `README.md`)

Структура проекта

```
lab4/
├── Matrix.h
├── Matrix.cpp
├── main.cpp
└── README.md
```

Сборка: Как пример, в командной строке:

```
g++ -std=c++17 -Wall -Wextra main.cpp Matrix.cpp -o lab4
```

компилятор не выдает предупреждений.

Git: три коммита

Итог выполнения работы:

Компиляция с -Wall -Wextra без предупреждений
Конструктор копирования делает глубокое копирование
operator= освобождает старую память перед копированием
operator= защищён от самоприсваивания
operator= возвращает Matrix&, цепочка d = e = a работает
Пустая матрица (0×0) обрабатывается корректно в обоих методах
README с описанием класса
Три осмысленных коммита в Git

Вопросы для самопроверки

1. Чем конструктор копирования отличается от оператора присваивания? Когда вызывается каждый из них?
2. Что произойдёт, если убрать защиту от самоприсваивания и выполнить a = a;?
3. Почему operator= возвращает Matrix&, а не void? Что сломается, если вернуть void?
4. Что такое поверхностное копирование и почему оно опасно для класса с динамической памятью?
5. В каком порядке вызываются деструкторы при выходе объектов из области видимости в main?

Домашнее задание

1. Рефакторинг: вынести работу с памятью в три приватных метода (см. методические указания).

Методические указанию к выполнению домашнего задания:

Цель рефакторинга: убрать следующие дублирования кода работы с памятью:

- Выделение (new int*[...] и цикл new int[...]) — в конструкторе с параметрами, в конструкторе копирования, в operator=.
- Освобождение (delete[] в цикле и delete[] data_) — в деструкторе и в operator=.
- Копирование значений (двойной цикл) — в конструкторе копирования и в operator=.

Почему дублирование плохо: при исправлении одной ошибки придётся править в нескольких местах, и легко забыть это сделать. Нужно вынести всю работу с памятью в **три приватных метода** и переписать соответственно конструкторы, деструктор и operator=.

Какие методы ввести

В приватной части Matrix.h объявить три метода:

```
void allocate(int rows, int cols);       // выделяет память
void copyFrom(const Matrix& other);      // копирует данные из other в *this
void freeMemory();                       // освобождает память и обнуляет поля
```

Назначение каждого метода:

Метод	**Что делает**	**Когда вызывается**
allocate(rows, cols)	Выделяет new int*[rows] и new int[cols] на каждую строку. **Требует:** rows > 0, cols > 0, data_ == nullptr. Присваивает rows_, cols_. **Значения не инициализирует.**	Из конструктора с параметрами и из copyFrom.
copyFrom(other)	Приводит *this к копии other. Если other пустая — оставляет *this пустой и выходит. Иначе — вызывает allocate и поэлементно копирует значения. **Требует:** *this уже пустой.	Из конструктора копирования и из operator=.
freeMemory()	Освобождает всю память (сначала строки, потом массив указателей), обнуляет data_, rows_, cols_. Безопасен для пустого объекта.	Из деструктора и из operator=.

Как переписать существующие методы:

Конструктор с параметрами:

- если rows <= 0 || cols <= 0 — оставить 0×0 и выйти;
- иначе вызвать allocate(rows, cols), затем заполнить нулями двойным циклом.

Конструктор копирования:

- в списке инициализации: data_(nullptr), rows_(0), cols_(0);
- в теле: один вызов copyFrom(other);.

Оператор присваивания:

```
Matrix& Matrix::operator=(const Matrix& other) {
    if (this == &other) return *this;    // защита от самоприсваивания
    freeMemory();                        // освободить старую память
    copyFrom(other);                     // скопировать новые данные
    return *this;                        // для цепочек
}
```

Деструктор: один вызов freeMemory();.

Подсказки по реализации

freeMemory():

- Начните с if (data_ != nullptr) — если объект уже пустой, ничего не делаем.
- Цикл for (int i = 0; i < rows_; ++i) delete[] data_[i];, потом delete[] data_;.
- Обязательно после delete[] data; присвойте data_ = nullptr; — это защита от висячего указателя.
- В конце — rows_ = 0; cols_ = 0; — приводим объект к пустой матрице.

allocate(rows, cols):

- Проверок на 0 быть не должно — этот метод вызывается только когда точно есть что выделять.
- Сначала data_ = new int*[rows];.
- Потом цикл for (int i = 0; i < rows; ++i) data_[i] = new int[cols];.
- В конце rows_ = rows; cols_ = cols;.
- Значения не инициализируются — это ответственность вызывающего (конструктор с параметрами заполнит нулями, copyFrom — копией).

copyFrom(other):

- Первая проверка: if (other.rows_ == 0 || other.cols_ == 0 || other.data_ == nullptr) return; — если источник пустой, *this остаётся 0×0.
- Иначе — allocate(other.rows_, other.cols_); и двойной цикл data_[i][j] = other.data_[i][j];.

Что должно получиться:

1. После рефакторинга все методы становятся короткими:

- конструктор с параметрами — проверка + allocate + заполнение нулями;
- конструктор копирования — один вызов copyFrom;
- operator= — 4 строки;
- деструктор — один вызов freeMemory.

2. Не должно быть двойных циклов копирования в трёх местах — только в copyFrom.

3. Цикл delete[] встречается только в freeMemory;

4. new int*[...] встречается только в allocate;

5. Демонстрационная программа работает как раньше.